

Antony Schutz

Semaine 3

Sommaire

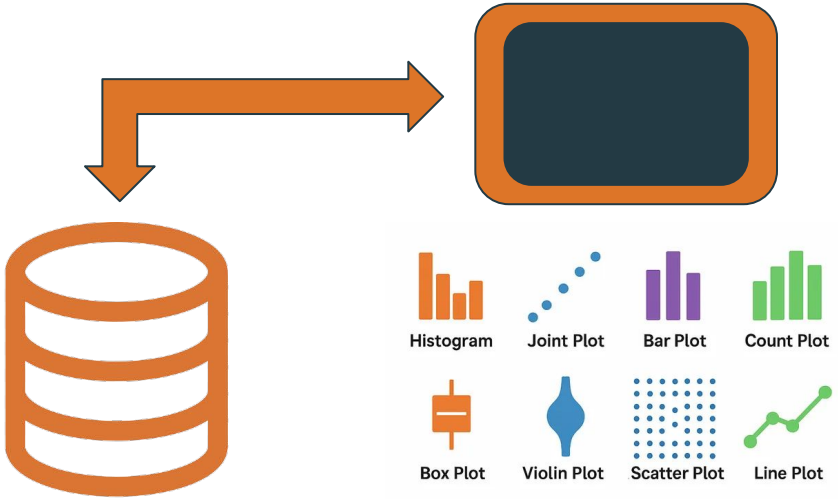
- Qu’est ce qu’une base de données?
- Les types de données
- Les requêtes de base
- SQLALCHEMY: initiation
- Optimisation de la mémoire et pourquoi?
- Veille technologique courte : Qu’est ce qu’un schéma de base de données?
- Conception d’une base de données : la méthode **MERISE**
- Les requêtes avancées
- POO et ORM

Qu’est ce qu’une base de données?

Une base de données (BDD) est un espace de stockage scalable optimisé pour le stockage et l’accès aux données.

Une base de données SQL est un type de BDD.

Une BDD est composée de tables



customer_id	first_name	last_name	phone	email	street	city	state	zip_code
1	Debra	Burks	NULL	debra.burks@yahoo.com	9273 Thorne Ave.	Orchard Park	NY	14127
2	Kasha	Todd	NULL	kasha.todd@yahoo.com	910 Vine Street	Campbell	CA	95008
3	Tameka	Fisher	NULL	tameka.fisher@aol.com	769C Honey Creek St.	Redondo Beach	CA	90278
4	Daryl	Spence	NULL	daryl.spence@aol.com	988 Pearl Lane	Uniondale	NY	11553
5	Charolette	Rice	(916) 381-6003	charolette.rice@msn.com	107 River Dr.	Sacramento	CA	95820
6	Lyndsey	Bean	NULL	lyndsey.bean@hotmail.com	769 West Road	Fairport	NY	14450
7	Latasha	Hays	(716) 986-3359	latasha.hays@hotmail.com	7014 Manor Station Rd.	Buffalo	NY	14215
8	Jacquline	Duncan	NULL	jacquline.duncan@yahoo.com	15 Brown St.	Jackson Heights	NY	11372
9	Genoveva	Baldwin	NULL	genoveva.baldwin@msn.com	8550 Spruce Drive	Port Washington	NY	11050
10	Pamelia	Newman	NULL	pamelia.newman@gmail.com	476 Chestnut Ave.	Monroe	NY	10950
11	Deshawn	Mendoza	NULL	deshawn.mendoza@yahoo.com	8790 Cobblestone Street	Monsey	NY	10952
12	Robby	Sykes	(516) 583-7761	robby.sykes@hotmail.com	486 Rock Maple Street	Hempstead	NY	11550
13	Lashawn	Ortiz	NULL	lashawn.ortiz@msn.com	27 Washington Rd.	Longview	TX	75604
14	Garry	Espinoza	NULL	garry.espinoza@hotmail.com	7858 Rockaway Court	Forney	TX	75126

Si on a 4 bits , on a 4 interrupteurs

0	0	0	0
0	0	0	1
0	0	1	0
0	0	1	1
0	1	0	0
0	1	0	1
0	1	1	0
0	1	1	1
1	0	0	0
1	0	0	1
1	0	1	0
1	0	1	1

Les types de données

Type	Nombre d'octets	Minimum	Maximum
TINYINT	1	-128	127
SMALLINT	2	-32768	32767
MEDIUMINT	3	-8388608	8388607
INT	4	-2147483648	2147483647
CHAR	1		

SQL Data Types for MySQL, SQL Server, and MS Access

Les types de données

Type	Pourquoi
`TINYINT`	indice, petit choix
`SMALLINT/MEDIUMINT/INT`	Grand choix
`VARCHAR`	Adresse, adresse mail, nom, prénom etc...
`BOOL`	vrai/faux
`FLOAT`	Nombre à virgule comme les prix, taille etc...
`TEXT`	Pour les longs textes (blog)

SQL Data Types for MySQL, SQL Server, and MS Access

Syntaxe de base pour lire

SELECT	SÉLECTIONNER
FROM	DEPUIS
AS	COMME
WHERE	OÙ
GROUP BY	GROUPER PAR
AND/OR	ET/OU
LIMIT	LIMITÉ À

Exemple

Table “client” :

identifiant	prenom	nom	ville
1	Pierre	Dupond	Paris
2	Sabrina	Durand	Nantes
3	Julien	Martin	Lyon
4	David	Bernard	Marseille
5	Marie	Leroy	Grenoble

```
SELECT ville FROM client
```

ville
Paris
Nantes
Lyon
Marseille
Grenoble

```
SELECT prenom, nom FROM client
```

prenom	nom
Pierre	Dupond
Sabrina	Durand
Julien	Martin
David	Bernard
Marie	Leroy

Dossier mini projet

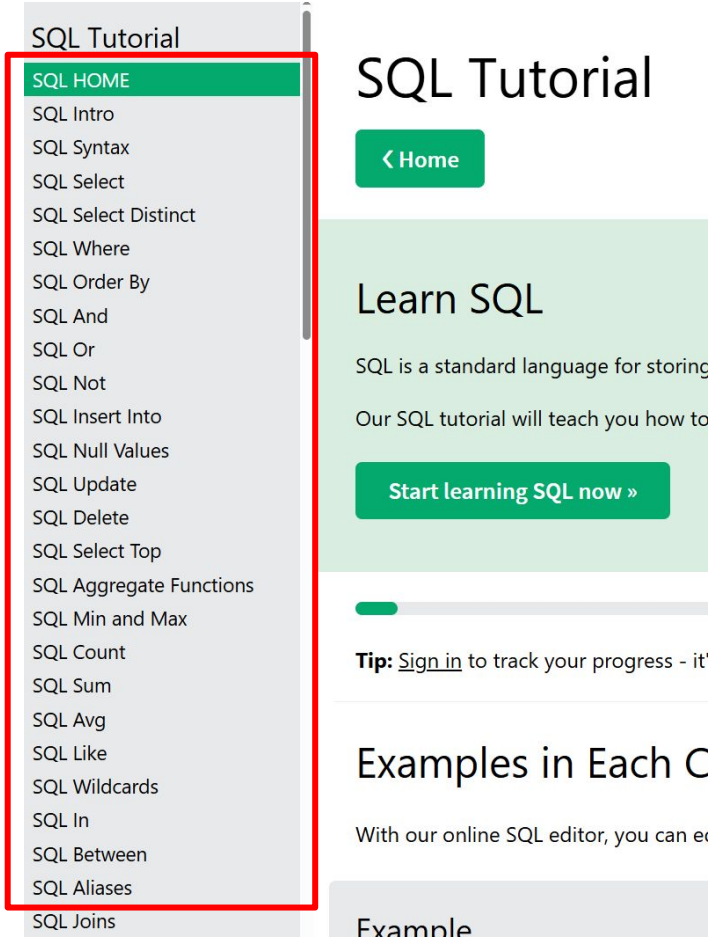
- Nouveau dossier
- environnement virtuel
- README.md
- notebook *.ipynb
- github (gitignore) commit and push

```
pip install sqlalchemy pandas tabulate
```

Entrainement

Faites les tutos et les exercices de :

[SQL Tutorial]



SQLAlchemy - 1

Téléchargez la base de données SAKILA

Utiliser SQLAlchemy pour se connecter à la base

Afficher le contenu de la table ACTOR



https://github.com/bradleygrant/sakila-sqlite3/blob/main/sakila_master.db

```python
from sqlalchemy import create_engine, text
import pandas as pd

# --- 1. Configuration et Connexion ---
DB_FILE = "sakila_master.db"
# Le moteur utilise le connecteur sqlite3 intégré
engine = create_engine(f"sqlite:///{DB_FILE}")

print(f"Connexion établie à la base de données : {DB_FILE}\n")

# --- 2. Requête SQL (La Jointure) ---
# Objectif : A REMPLIR
# La requête se termine avec un ;

SQL_QUERY = text(""" REQUETE; """)

# --- 3. Exécution et Récupération des Résultats ---
try:
    with engine.connect() as connection:
        # Exécuter la requête
```



```python
try:
    with engine.connect() as connection:
        # Exécuter la requête
        result = connection.execute(SQL_QUERY)

        # Récupérer les noms des colonnes pour les utiliser dans Pandas
        column_names = list(result.keys())

        # Récupérer toutes les lignes (Fetch all)
        data = result.fetchall()

    # --- 4. Affichage des Résultats (avec Pandas pour une belle table) ---
    # Pandas est souvent utilisé en Data Science pour l'affichage de données
    df = pd.DataFrame(data, columns=column_names)

    print("--- Résultats de la requête ---")
    print(df.to_markdown(index=False)) # to_markdown est utile pour l'affichage en terminal
    print("-------------------------------------------")

except Exception as e:
    print(f"Une erreur s'est produite lors de l'exécution de la requête : {e}")
```

petits exercices

MEMO : énoncé sur github

1_exercices_simple.md

Puis

```
pip install seaborn
```

OPCO - Pandas - Seaborn

2_exercices_seaborn.md

Optimisation de la mémoire et pourquoi?

En IA on est souvent confronté à des volumes de données immenses.

Il y a deux stratégies:

- optimiser la taille en mémoire
- optimiser la vitesse des requêtes

Ici on s'intéresse à la taille en mémoire



Exemple

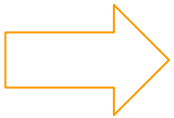
Type	Nombre d'octets
TINYINT	1
SMALLINT	2
MEDIUMINT	3
INT	4
CHAR	1

Table d'origine

Table utilisateur

ID	Nom	Pays
1	John	usa
2	Mike	usa
3	François	fr

3 chaînes de caractères



Double table

Table utilisateur

ID	Nom	Pays_ID
1	John	0
2	Mike	0
3	François	1

1 entier

Table Pays

Pays_ID	Pays
0	usa
1	fr

2 chaînes de caractère

Notion de clé primaire et clé étrangère

Les clés sont essentielles afin de retrouver les liens entre les tables

ID	Nom	Pays_ID
1	John	0
2	Mike	0
3	François	1

Pays_ID	Pays
0	usa
1	fr



	type	clé
ID	int	primaire
Nom	varchar	
Pays_ID	int	étrangère

	type	clé
Pays_ID	int	primaire
Pays	varchar	

Les alias

Pays_ID
0
0
1

Quand on a le même nom sur plusieurs table

```
SELECT Pays_ID FROM user

SELECT Pays_ID FROM pays
```

on peut changer leur nom avec un alias **AS**:

U_P_ID
0
0
1

```
SELECT Pays_ID AS U_P_ID FROM user

SELECT Pays_ID AS P_P_ID FROM pays
```

on peut aussi, et c'est **important** pour la suite, donner un alias **AS** à la table:

```
SELECT U.Pays_ID FROM user AS U
SELECT U.Pays_ID FROM user U
```

AS peut être remplacé par un espace

user

ID	Nom	Pays_ID
1	John	0
2	Mike	0
3	François	1

pays

Pays_ID	Pays
0	usa
1	fr

Mais comment retrouver toutes les données?

utilisateur

ID	Nom	Pays
1	John	0
2	Mike	0
3	François	1

Pays

ID	Pays
0	usa
1	fr

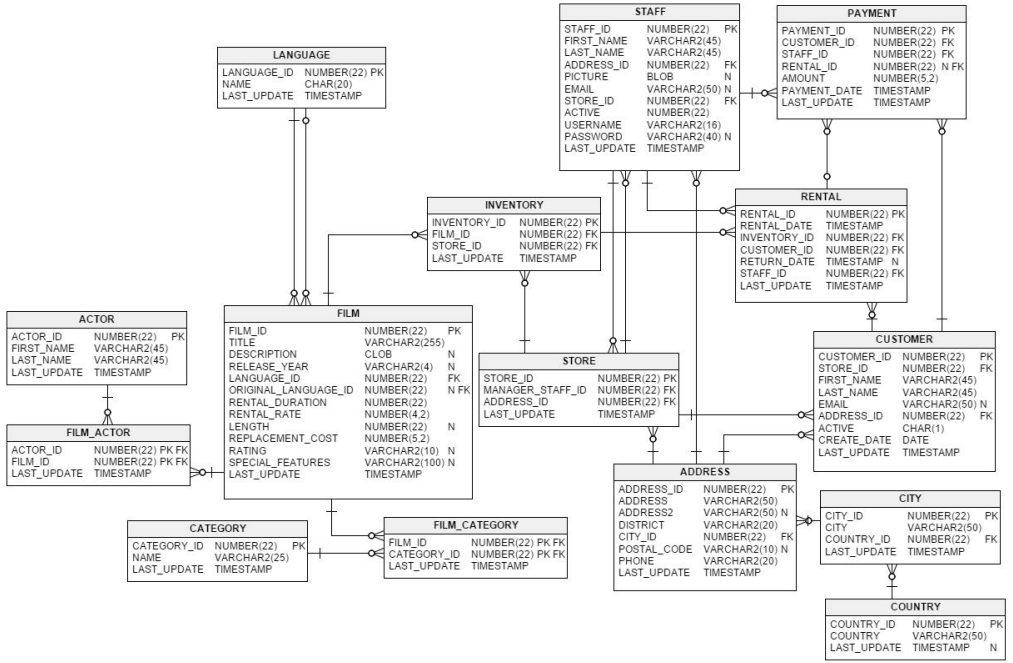
```
SELECT U.ID, U.Nom, P.Pays  FROM utilisateur AS U JOIN Pays AS P ON U.Pays=P.ID
```

ID	Nom	Pays
1	John	usa
2	Mike	usa
3	François	fr

La base de données SAKILA?

Analysez le schéma de la base de données sakila et effectuez les recherches suivantes :

3_exercices_jointures.md



https://github.com/bradleygrant/sakila-sqlite3

Challenges à la maison

https://www.sql-practice.com/

https://mystery.knightlab.com/

https://sql-island.informatik.uni-kl.de/

https://www.sqlnoir.com/

https://sqlpd.com/

Requêtes imbriquées

```sql
SELECT
    title,
    length
FROM
    film
WHERE
    length > (
        -- SOUS-REQUÊTE (Étape 1 : Trouve la durée moyenne)
        SELECT AVG(length)
        FROM film
    )
ORDER BY
    length DESC; LIMIT 10;
```

Trouvez le **titre** et la **durée** des films (`title`, `length` de la table `film`) dont la durée est **supérieure à la durée moyenne** de *tous* les films présents dans la base de données.

- **Logique :** La sous-requête doit d'abord calculer la moyenne (`AVG(length)`) et retourner une **valeur unique** pour que la requête principale puisse utiliser l'opérateur de comparaison (`>`).

La sous requête se place sous forme de condition et est encadré par des parenthèse.

4_exercices_subrequest.md

Méthode MERISE

Conception de BDD : méthode MERISE

MCD - Modèle conceptuel des données :

- les informations redondantes sont séparées des tables de fait ou de la table de fait (rectangles)
- définit les relations entre les tables (ellipses) et les cardinalités entre le tables

MLD - Modèle logique des données :

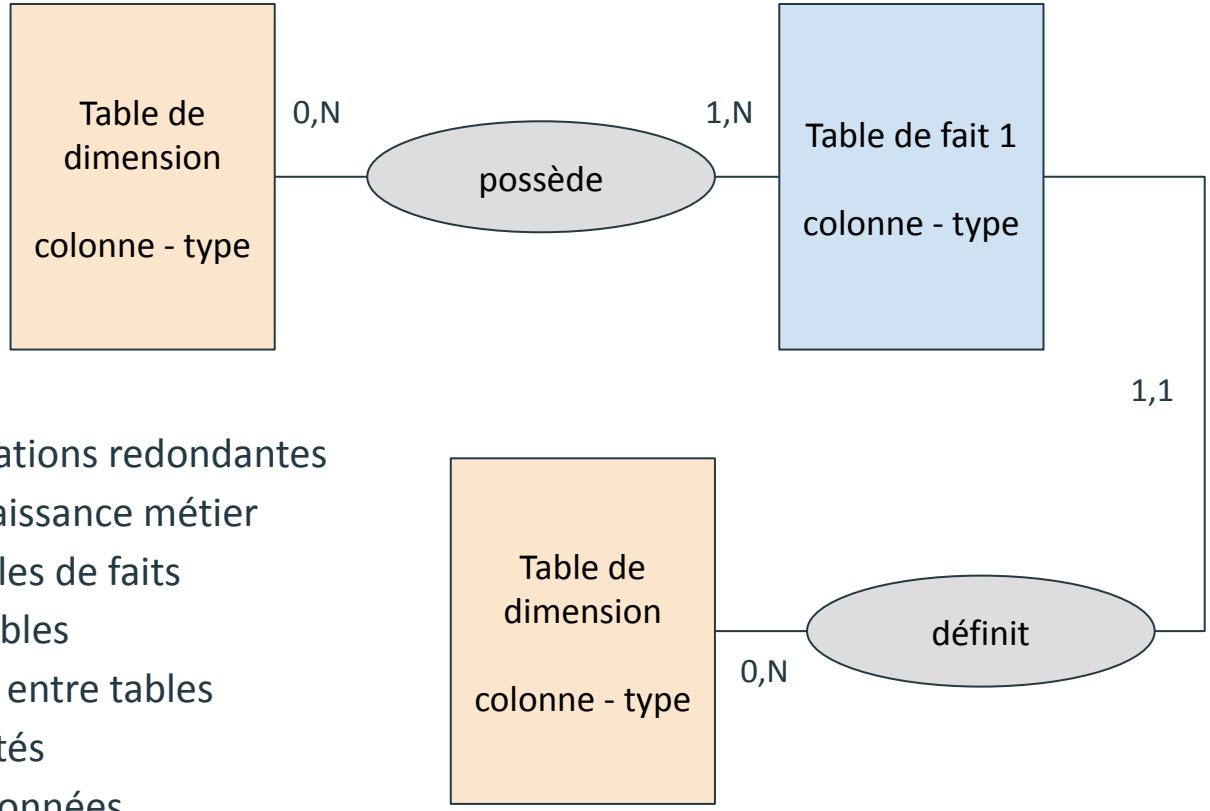
- définit les clés étrangères, primaire et les colonnes

MPD - Modèle physique des données :

- le modèle finale prêt à être implémenter

Zéro à Un	(0, 1)	**Zero or One**	L'entité participe **au maximum une fois** et peut ne pas participer du tout.	Un Client peut avoir **zéro ou un** Téléphone Fixe.
Un à Un	(1, 1)	**One and Only One**	L'entité participe **exactement une fois** (participation obligatoire et unique).	Un Pays a **exactement un** Drapeau Officiel.
Zéro à Plusieurs	(0, n) ou (0, *)	**Zero or More**	L'entité peut participer **plusieurs fois** ou ne pas participer (le cas le plus fréquent).	Un Client passe **zéro ou plusieurs** Commandes.
Un à Plusieurs	(1, n) ou (1, *)	**One or More**	L'entité participe **au moins une fois** et peut participer plusieurs fois (participation obligatoire).	Un Professeur doit gérer **au moins un** Cours (et plusieurs).
Plusieurs à Plusieurs	(n, m) ou (m, n)	**Many to Many**	Les deux entités peuvent participer **plusieurs fois** l'une à l'autre (se résout avec une table d'association).	Un Film à **plusieurs** Acteurs, et un Acteur joue dans **plusieurs** Films.

MCD



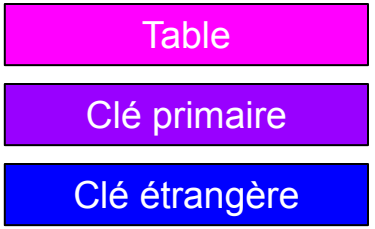
ordre des opérations:

- on identifie les informations redondantes
- on applique une connaissance métier
- on définit la ou les tables de faits
- on définit les autres tables
- on définit les relations entre tables
- on définit les cardinalités
- on décrit le type des données

MLD

un tableau ou des lignes qui définissent :

- le nom de chaque table
- la clé primaire
- les clés étrangères



Clé Primaire (PK) : C'est l'identifiant unique qui garantit que chaque ligne d'une table est distincte et peut être référencée sans ambiguïté.

Clé Étrangère (FK) : C'est un champ dans une table qui pointe vers la Clé Primaire d'une *autre* table, créant ainsi la relation entre les données.

MPD

MCD + MLD fusionné

On retrouve les tables définitives avec les clés primaires <PK> et étrangères <FK> et les types de données au sens du code, entier devient int etc …

Les relations et les cardinalités ne sont plus affichées

Table d’association

Pour certaines relations plus complexe on peut avoir besoin de table d’association, cela permet de gérer les relations many to many

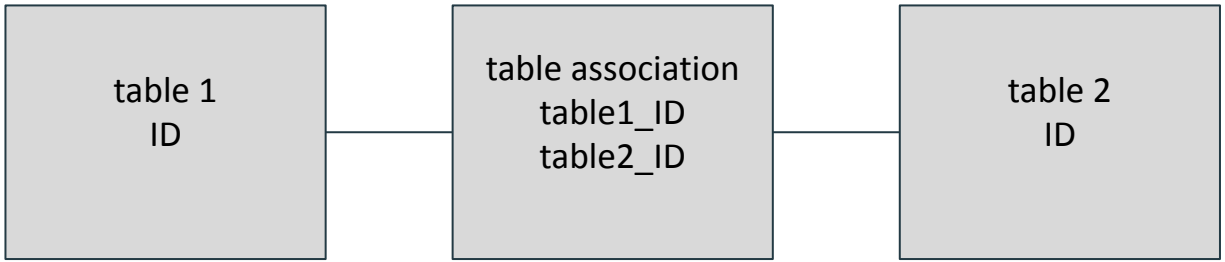


Table d'association

ID	NOM
1	Sadi
2	K
3	Julie
4	Tim

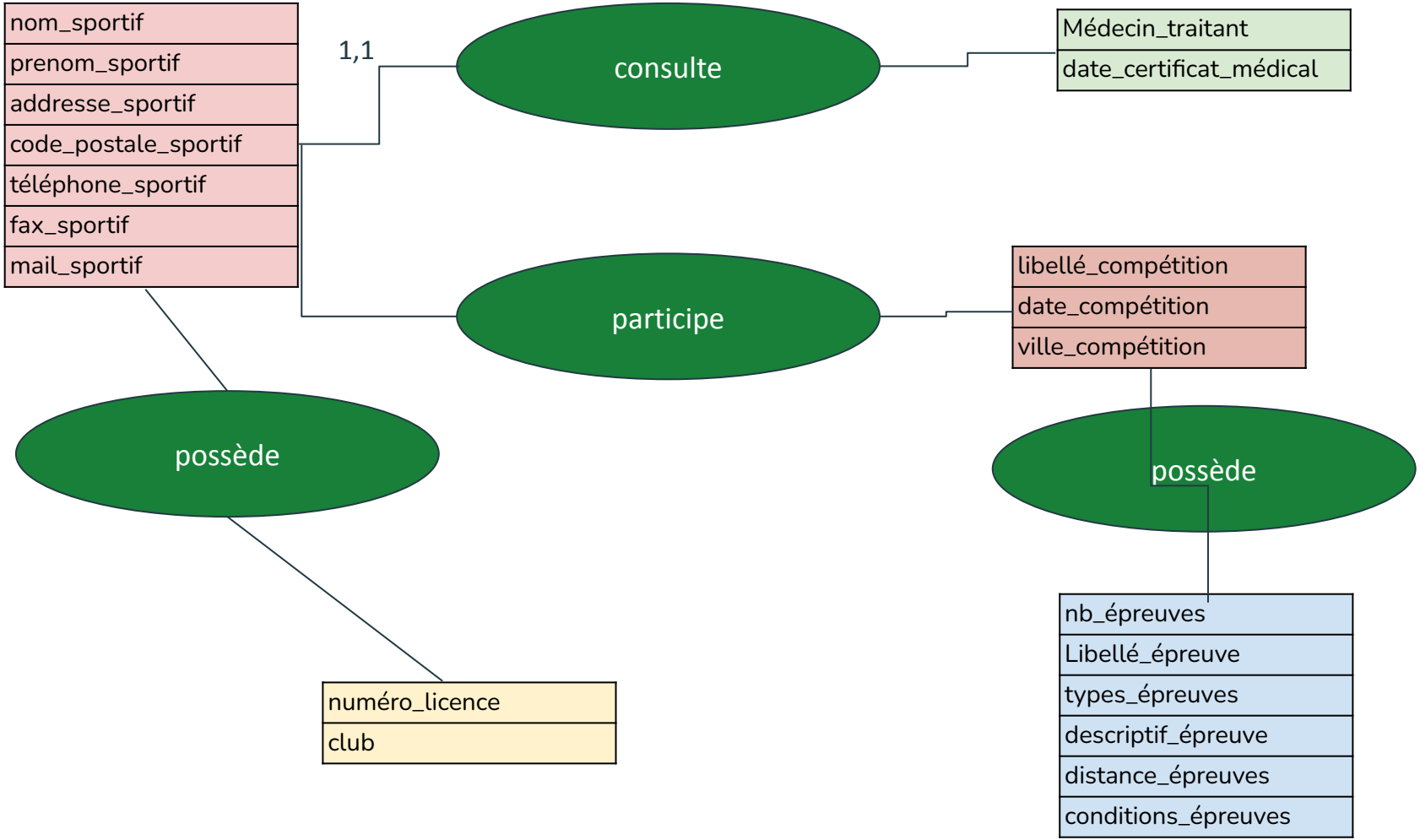
ID	FK_NOM	FK_FRUIT	Nombre
1	1	1	3
2	1	2	2
3	2	3	2
4	2	4	1
5	2	2	1
6	3	3	1

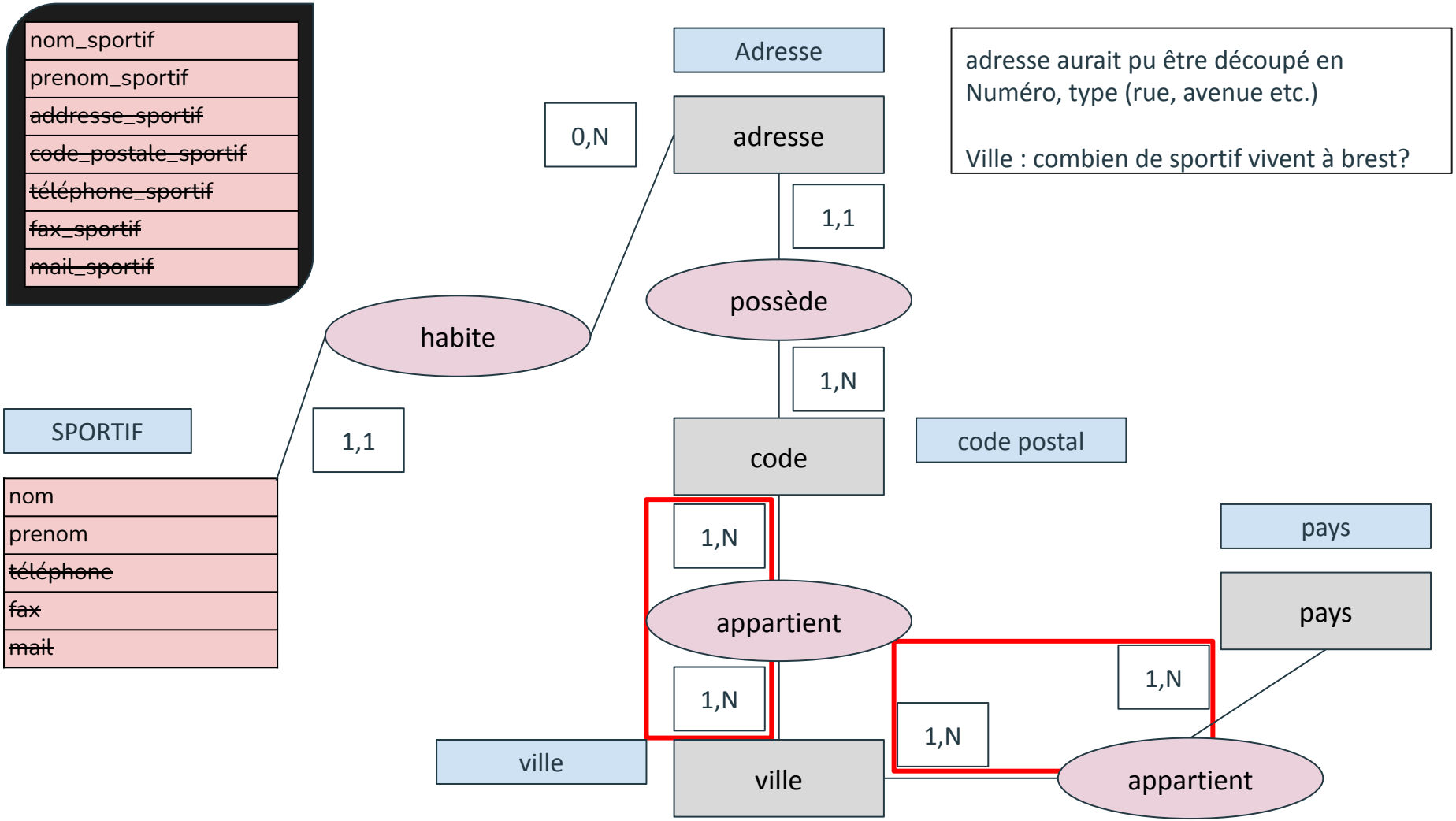
ID	FRUIT
1	pomme
2	orange
3	fraise
4	kiwi

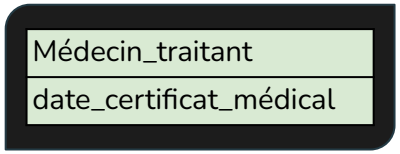
Sadi possède 3 pommes et deux oranges, Kimberly possède 2 fraises, un kiwi et une orange, Tim ne possède rien du tout et Julie à une fraise

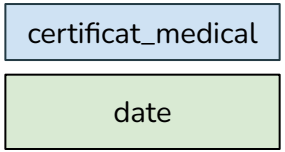
Libellé	Type	Taille	Description
nom_sportif	chaîne de caractères	30	nom de famille du sportif
prenom_sportif	chaîne de caractères	30	prénom du sportif
addresse_sportif	chaîne de caractères	50	adresse du sportif
code_postale_sportif	chaîne de caractères	15	code postale de l'adresse du sportif
téléphone_sportif	chaîne de caractères	15	téléphone du sportif
fax_sportif	chaîne de caractères	15	fax du sportif
mail_sportif	chaîne de caractères	50	mail du sportif
numéro_dossard	int		numéro de dossard du sportif pour la compétition
numéro_licence	chaîne de caractères	20	numéro de la licence
club	chaîne de caractères	50	nom du club sportif
Médecin_traitant	chaîne de caractères	50	nom du médecin traitant
date_certificat_médical	date		date
libellé_compétition	chaîne de caractères	100	nom de la compétition
date_compétition	date		date de la compétition
ville_compétition	chaîne de caractères	50	ville de la compétition
nb_épreuves	int		nombre d'épreuves
Libellé_épreuve	chaîne de caractères	100	intitulé de l'épreuve
types_épreuves	chaîne de caractères	100	types d'épreuves
descriptif_épreuve	chaîne de caractères	100	description de l'épreuve
distance_épreuves	float		distances de l'épreuve
conditions_épreuves	chaîne de caractères	100	conditions de l'épreuve

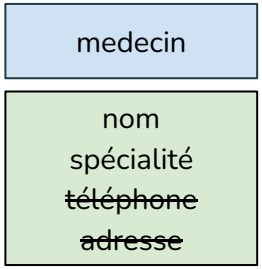
Libellé	Type	Taille	Description
nom_sportif	chaîne de caractères	30	nom de famille du sportif
prenom_sportif	chaîne de caractères	30	prénom du sportif
addresse_sportif	chaîne de caractères	50	adresse du sportif
code_postale_sportif	chaîne de caractères	15	code postale de l'adresse du sportif
téléphone_sportif	chaîne de caractères	15	téléphone du sportif
fax_sportif	chaîne de caractères	15	fax du sportif
mail_sportif	chaîne de caractères	50	mail du sportif
numéro_dossard	int		numéro de dossard du sportif pour la compétition
numéro_licence	chaîne de caractères	20	numéro de la licence
club	chaîne de caractères	50	nom du club sportif
Médecin_traitant	chaîne de caractères	50	nom du médecin traitant
date_certificat_médical	date		date
libellé_compétition	chaîne de caractères	100	nom de la compétition
date_compétition	date		date de la compétition
ville_compétition	chaîne de caractères	50	ville de la compétition
nb_épreuves	int		nombre d'épreuves
Libellé_épreuve	chaîne de caractères	100	intitulé de l'épreuve
types_épreuves	chaîne de caractères	100	types d'épreuves
descriptif_épreuve	chaîne de caractères	100	description de l'épreuve
distance_épreuves	float		distances de l'épreuve
conditions_épreuves	chaîne de caractères	100	conditions de l'épreuve

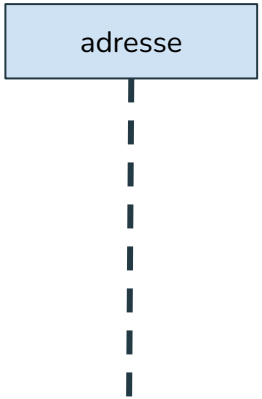

nom_sportif
prenom_sportif
addresse_sportif
code_postale_sportif
téléphone_sportif
fax_sportif
mail_sportif
1,1
consulte
Médecin_traitant
date_certificat_médical
participe
libellé_compétition
date_compétition
ville_compétition
possède
possède
numéro_licence
club
nb_épreuves
Libellé_épreuve
types_épreuves
descriptif_épreuve
distance_épreuves
conditions_épreuves

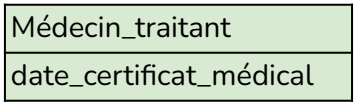

nom_sportif
prenom_sportif
~~addresse_sportif~~
~~code_postale_sportif~~
~~téléphone_sportif~~
~~fax_sportif~~
~~mail_sportif~~
Adresse
adresse aurait pu être découpé en
Numéro, type (rue, avenue etc.)
Ville : combien de sportif vivent à brest?
0,N
adresse
1,1
possède
habite
1,N
SPORTIF
1,1
code
code postal
nom
prenom
~~téléphone~~
~~fax~~
~~mail~~
1,N
pays
appartient
pays
1,N
1,N
1,N
ville
ville
appartient


Médecin_traitant
date_certificat_médical


certificat_medical
date


medecin
nom
spécialité
~~téléphone~~
~~adresse~~


adresse


Médecin_traitant
date_certificat_médical

Veille technologique courte

Qu’est ce qu’un schéma de base de données?

- Schéma en étoiles
- Schéma en flocon de neiges
- Schéma en Galaxie

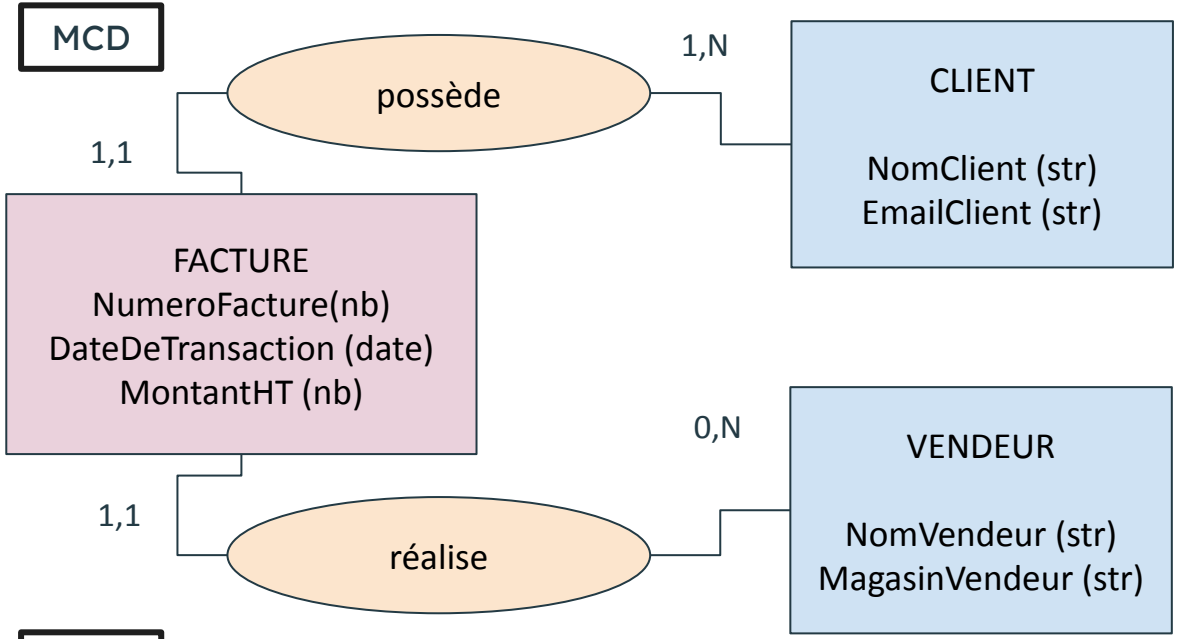


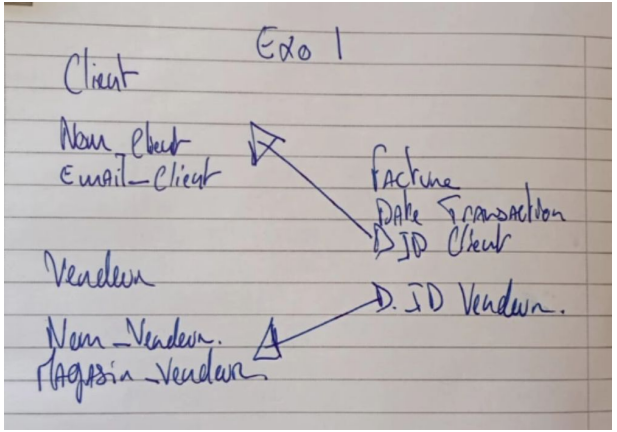
exercices

Conception MERISE MCD/MLD/MPD des 3 tableaux suivants :

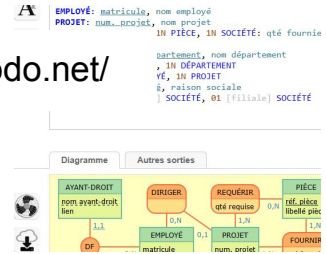
5_exercices_merise.md

MCD





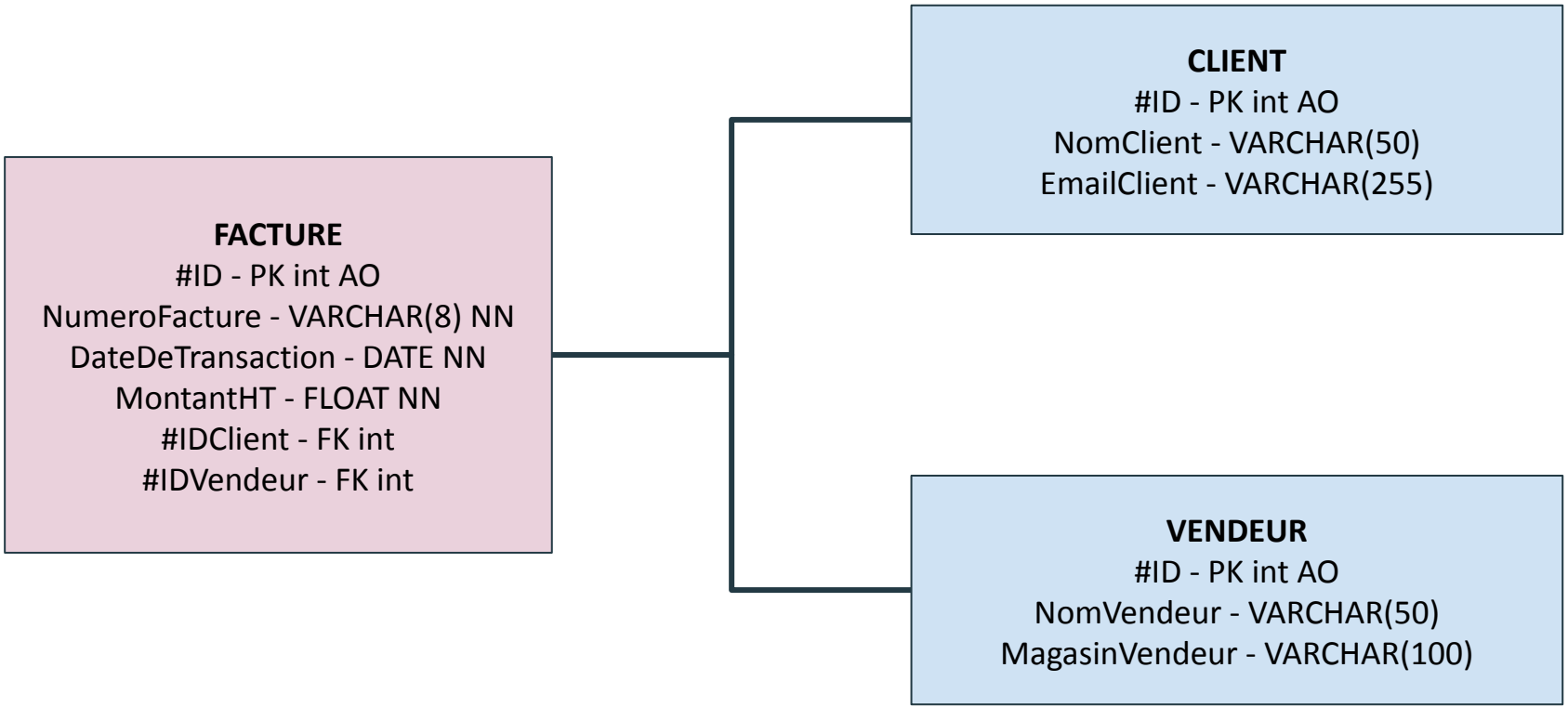
https://www.mocodo.net/



MLD

Primary Key	Client	ID (int), NomClient (VARCHAR 50), EmailClient (VARCHAR 255)
Foreign Key	Vendeur	ID (int), NomVendeur (VARCHAR 50), MagasinVendeur (VARCHAR 100)
	Facture	ID (int), NumeroFacture (VARCHAR 8), DateDeTransaction (DATE), MontantHT (DECIMAL [10,2]), IDClient (Client.ID, int), IDVendeur (Vendeur.ID, int)

MPD


FACTURE
#ID - PK int AO
NumeroFacture - VARCHAR(8) NN
DateDeTransaction - DATE NN
MontantHT - FLOAT NN
#IDClient - FK int
#IDVendeur - FK int
CLIENT
#ID - PK int AO
NomClient - VARCHAR(50)
EmailClient - VARCHAR(255)
VENDEUR
#ID - PK int AO
NomVendeur - VARCHAR(50)
MagasinVendeur - VARCHAR(100)

SQL Alchemy ORM



SQLAlchemy est une bibliothèque Python qui permet de communiquer avec une base de données relationnelle (PostgreSQL, SQLite, MySQL, etc.) en utilisant du code Python orienté objet. Son ORM (Object-Relational Mapper) permet de représenter les tables SQL sous forme de classes Python.

https://docs.sqlalchemy.org/en/20/orm/

CLASSE
(partie théorique d'un objet)

OBJET
C'est un objet, c'est une chose qui a des variables (attributs) et qui a des fonctions (méthodes)

Objet Doberman
age = 1 etc…
Méthodes:
marcher, courir, nager, sauter

instance

Classe CHIEN
Attributs:
age, nom, taille, poids, vitesse, race
Méthodes:
marcher, courir, nager, sauter

Doberman = new CHIEN(age=1, nom="dd")

EXEMPLE TYPE EN PYTHON = STR

Vocabulaire (ORM)

Terme Merise	SQLAlchemy	Description
Entité	Classe Python	Représente une table
Attribut	`Column`	Représente une colonne dans la table
Identifiant	`primary_key=True`	Clé primaire
Association	`ForeignKey` + `relationship`	Lien entre deux entités

Définir une table avec SQLAlchemy




```python
from sqlalchemy import Column, Integer, String, ForeignKey
from sqlalchemy.orm import declarative_base, relationship


Base = declarative_base()


class Client(Base):
    __tablename__ = 'clients'
    # clé primaire
    id = Column(Integer, primary_key=True)
    nom = Column(String)
    # relation 1-N
    commandes = relationship("Commande", back_populates="client")


class Commande(Base):
    __tablename__ = 'commandes'
    id = Column(Integer, primary_key=True)
    montant = Column(Integer)
    # clé étrangère
    client id = Column(Integer, ForeignKey('clients.id'))
    client = relationship("Client", back_populates="commandes")
```

Les relations bidirectionnelles (avec `back_populates`) permettent d'accéder facilement aux données dans les deux sens :

- Client → Commandes
- Commande → Client

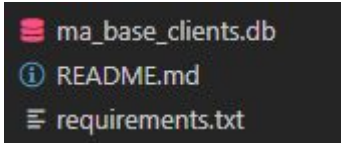
Connection à une base de données
Création de la base de données

```python
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

# 1. Connexion à une base de données SQLite :
# L'argument 'sqlite:///ma_base_clients.db' crée un fichier
'ma_base_clients.db'
# dans le répertoire courant s'il n'existe pas.
engine = create_engine("sqlite:///ma_base_clients.db", echo=True)

# 2. Création des tables dans la base de données
# Cette ligne regarde toutes les classes qui héritent de Base (Client,
Commande)
# et crée les tables SQL correspondantes (si elles n'existent pas déjà).
Base.metadata.create_all(engine)

# 3. Configuration de la session :
# Lie la session à notre moteur (engine) pour pouvoir interagir avec la BDD.
Session = sessionmaker(bind=engine)
session = Session()
```



Manipuler les données avec l'ORM

Création d'un client et d'une commande

```
nouveau_client = Client(nom="Alice")
session.add(nouveau_client)
session.commit()


commande = Commande(montant=200,
client=nouveau_client)
session.add(commande)
session.commit()
```

Lecture d'une commande avec filtrage et limite

```
# Lire toutes les commandes d'un client
client = session.query(Client).filter_by(nom="Alice").first()
print("-"*30)
print("--- Résultat de la requête ---")
for cmd in client.commandes:
    print("-"*30)
    print(cmd.montant)
    print("-"*30)
```

Manipuler les données avec l'ORM

modification d’une commande

```
commande_a_modifier =
session.query(Commande).filter_by(id=1).first()

# Modifier un montant
commande_a_modifier.montant = 300
session.commit()
```

Suppression d’une commande

```
# Supprimer une commande
session.delete(commande_a_modifier)
session.commit()
```

Fermer proprement la session

```
session.dispose()
```

Manipuler les données avec l'ORM

Insérer plusieurs commandes

```
# Création des nouveaux objets Commande en Python
nouvelle_commande_1 = Commande(montant=50, client=client_existant)
nouvelle_commande_2 = Commande(montant=150, client=client_existant)
nouvelle_commande_3 = Commande(montant=25, client_id=client_existant.id)
# On peut aussi utiliser directement le client_id

# 1. Ajout des trois objets à la session en une seule fois (add_all)
session.add_all([
      nouvelle_commande_1,
      nouvelle_commande_2,
      nouvelle_commande_3
])
# 2. Le commit unique exécute toutes les commandes INSERT en base de données.
session.commit()
```

Manipuler les données avec l'ORM

Lire toutes les commandes avec jointures

```
# Récupération de tous les objets Commande de la table
toutes_les_commandes = session.query(Commande).all()

print(f"Nombre total de commandes trouvées : {len(toutes_les_commandes)}")

# Itérer sur la liste des objets récupérés
for commande in toutes_les_commandes:
    print(f"- Commande ID: {commande.id}, Montant: {commande.montant}, Client ID:
{commande.client_id}, Client nom: {commande.client.nom}")
```

Bonne pratique utilisation d'un try / except

```
    # Le bloc 'with' garantit la fermeture de la session,
    # même en cas d'erreur.
    try:
        # 1. Création de la session (entre dans le bloc 'with')
        with Session() as session:
---
            print(f"\nCommandes de {client_neo.nom} :")
            for cmd in commandes_neo:
                print(f"  - ID Commande: {cmd.id}, Montant: {cmd.montant}")

    except SQLAlchemyError as e:
        # Si une erreur se produit (par exemple, si la BDD est inaccessible),
        # le 'rollback' est exécuté.
        session.rollback()
        print(f"\n❌ ERREUR DE TRANSACTION : Opération annulée. Détail: {e}")

    finally:
        # La session est automatiquement fermée après le bloc 'with'.
        print("\n--- Fin de la transaction (Session fermée) ---")
```